

sell sell marketplace updates:

SHAZ EJ



Given where PickPic/SellSell is today, I would avoid spending the next week on new features. The biggest value is making the existing marketplace intelligent and reliable before adding more complexity.

Week Goal

By the end of the week, Fahad should be able to type:

- Black Mercedes S400
- Toyota Camry under 3000 KWD
- iPhone 15 Pro 256GB
- White sofa in Doha
- Furnished apartment in Salmiya

and receive highly relevant results.

Day 1 - Full Search Audit

Objective

Understand exactly how listings flow from seller to buyer search.

Tasks

- Trace listing creation
- Trace moderation
- Trace structuring job
- Trace embedding generation
- Trace Qdrant indexing
- Trace search pipeline
- Document gaps

Deliverable

Search Architecture Document

Questions answered:

- Where is color stored?
 - Where is model stored?
 - Where is condition stored?
 - What reaches Qdrant?
 - What reaches SQL?
-

Day 2 - Attribute Extraction Upgrade

Objective

Improve AI structuring quality.

Tasks

Enhance extraction for:

Vehicles:

- Brand
- Model
- Year
- Color
- Fuel Type
- Transmission

Electronics:

- Brand
- Model
- Storage
- RAM

Real Estate:

- Bedrooms
- Bathrooms
- Furnished
- Area

Deliverable

Structured JSON quality > 90%

Example:

```
{  
  "brand": "Mercedes",  
  "model": "S400",  
  "color": "Black",  
  "year": 2021  
}
```

Day 3 - Qdrant Payload Improvement



Objective

Store searchable attributes in vector metadata.

Tasks

Add payload fields:

```
{
  "category": "vehicle",
  "brand": "Mercedes",
  "model": "S400",
  "color": "Black",
  "condition": "Used"
}
```

Deliverable

Every indexed listing contains rich metadata.

Day 4 - Buyer Search Improvement

Objective

Make buyer intent extraction smarter.

Tasks

Support:

- black mercedes s400
- cheap toyota camry
- iphone under 200 kwd
- furnished apartment

Extract:

```
{
  "brand": "Mercedes",
  "model": "S400",
  "color": "Black"
}
```

Deliverable

Intent extraction accuracy > 90%.



Day 5 - Hybrid Ranking Engine

Objective

Improve ranking quality.

Current

SQL
OR
Qdrant

Target

SQL
+
Qdrant
+
Attribute Match Score
+
AI Score

Deliverable

Black Mercedes ranks above white Mercedes.

Day 6 - End-to-End Testing

Objective

Test with realistic marketplace data.

Create Test Listings

Vehicles:

- Black Mercedes S400
- White Mercedes S400
- Toyota Camry

Electronics:

- iPhone 15 Pro
- Samsung S24

Property:



- Furnished Apartment
- Unfurnished Apartment

Run Tests

Search:

- black mercedes
- white mercedes
- iphone 256gb
- furnished apartment

Deliverable

Search quality report.

Day 7 - Fahad Demo Day

Objective

Show visible AI intelligence.

Demo Scenarios

Fahad types:

I want a black Mercedes S400

Result:

- Black Mercedes S400 first
-

Fahad types:

iPhone 15 Pro 256GB

Result:

- Correct storage variant first
-

Fahad types:



Furnished apartment in Salmiya

Result:

- Furnished apartments first
-

Expected Outcome After 1 Week

Before

- Search accuracy: ~75-85%
- Attribute understanding: inconsistent
- Color filtering: weak
- Ranking: basic

After

- Search accuracy: 90-95%
- Attribute extraction: strong
- Qdrant metadata: rich
- Ranking: attribute-aware
- Fahad's "black Mercedes S400" use case works reliably

This week would move PickPic from a marketplace with AI features to an AI-first marketplace where the search experience becomes the main selling point.



Yes. We should make it **tiny task by tiny task**, and after every task AntigraVity must run Docker + browser test.

Use this as the implementation prompt:

We will implement the buyer AI search upgrade in very small functions only.

Important rules:

1. Do not make large changes.
2. Do not change Prisma schema unless I approve separately.
3. After every small task, run Docker checks.
4. After every small task, test in browser.
5. Commit only after browser test passes.
6. Keep all changes compatible with the current Docker setup.
7. Use the existing RunPod/Ollama configuration. Do not switch to local Ollama.

Main goal:

Make this buyer search reliable:

“I want a black Mercedes S400”

The correct result should prioritize black Mercedes S400 over white, silver, or unrelated Mercedes listings.

Implementation must be divided into very small functions.

Phase 1: Audit Current Search

Task 1.1

Create a small trace/debug helper function:

`traceSearchQuery(query)`

It should log:

- original query
- extracted intent
- SQL filters
- Qdrant filters
- matched product IDs
- final ranked product IDs

Browser test:

- Start Docker
- Open site
- Login
- Search “black Mercedes S400”



- Confirm logs appear correctly

Do not modify ranking yet.

Task 1.2

Create a small helper:

`normalizeColor(value)`

It should normalize:

- black
- white
- silver
- grey/gray
- red
- blue
- green
- gold
- brown
- beige

Browser test:

- Search black Mercedes S400
- Confirm black is extracted/normalized

Task 1.3

Create a helper:

`extractAttributeTokens(query)`

It should detect:

- color
- brand
- model
- year
- price words
- location words

For now keep it simple and deterministic.

Browser test:

- black Mercedes S400
- white Toyota Camry
- iPhone 15 Pro 256GB
- furnished apartment in Salmiya



Task 1.4

Create a helper:

`mergeAllIntentWithRuleIntent(aiIntent, ruleIntent)`

Rule-based extracted fields should fill missing AI intent fields.

Browser test:

- Search black Mercedes S400
- Confirm final intent contains color black, brand Mercedes, model S400

Phase 2: Improve Qdrant Payload Without Schema Change

Task 2.1

Create helper:

`buildSearchableAttributes(product)`

It should collect attributes from:

- `product.title`
- `product.description`
- `product.aiDescription`
- `product.attributes JSON`
- `vehicleDetails` if available
- `propertyDetails` if available

Browser test:

- Open existing listings
- Confirm helper produces searchable attributes

Task 2.2

Update Qdrant payload builder to include:

- `normalizedColor`
- `brand`
- `model`
- `category`
- `condition`
- `location`
- `price`
- `searchableAttributes`

Browser test:

- Create one black Mercedes S400 listing
- Wait for indexing
- Confirm Qdrant payload contains color black



Task 2.3

Add safe fallback when attributes are missing.

Do not fail indexing if color/brand/model is missing.

Browser test:

- Create listing with only title
- Confirm listing still publishes and indexes

Phase 3: Improve Buyer Search Ranking

Task 3.1

Create helper:

`scoreAttributeMatch(intent, product)`

Scoring:

- exact color match: +30
- exact brand match: +25
- exact model match: +25
- category match: +15
- location match: +10
- price match: +10

Browser test:

- Create black Mercedes S400
- Create white Mercedes S400
- Search black Mercedes S400
- Confirm black one ranks first

Task 3.2

Create helper:

`explainSearchMatch(intent, product)`

Return simple reasons:

- Matches color black
- Matches brand Mercedes
- Matches model S400

Browser test:

- Search black Mercedes S400
- Confirm result has match explanation in logs or UI debug output



Task 3.3

Update final ranking to combine:

- SQL match score
- Qdrant semantic score
- attribute match score
- freshness score if already available

Browser test:

- Search black Mercedes S400
- Confirm correct order

Phase 4: Avoid Bad Qdrant Skipping

Task 4.1

Find current logic where Qdrant is skipped if SQL returns enough results.

Create helper:

```
shouldRunVectorSearch(intent, sqlResultCount)
```

Return true when:

- query has color
- query has model
- query has brand
- query has detailed attributes
- SQL result count is low
- query is natural language

Browser test:

- Search black Mercedes S400
- Confirm Qdrant still runs even if SQL finds results

Task 4.2

Merge SQL and Qdrant results with de-duplication.

Create helper:

```
mergeSearchResults(sqlResults, qdrantResults)
```

Browser test:

- Search Mercedes
- Confirm no duplicate listings appear

Phase 5: Test Dataset



Task 5.1

Create or use browser to add these listings:

1. Black Mercedes S400
2. White Mercedes S400
3. Silver Mercedes S400
4. Black Toyota Camry
5. iPhone 15 Pro 256GB
6. iPhone 15 Pro 128GB
7. Furnished apartment in Salmiya
8. Unfurnished apartment in Salmiya

Browser test:

- Confirm all listings publish successfully

Task 5.2

Run buyer searches:

- black Mercedes S400
- white Mercedes S400
- Mercedes S400
- black Toyota
- iPhone 15 Pro 256GB
- furnished apartment Salmiya

Browser test:

- Confirm correct listings rank first

Phase 6: Final QA

Run:

```
docker compose up -d
docker compose ps
docker compose logs app --tail=100
npm run typecheck
npx prisma validate
```

Then browser test:

1. Register/login
2. Add listing
3. Search listing
4. Verify listing page
5. Verify buyer AI search
6. Verify no console errors

Final deliverable:



Provide:

- files changed
- functions added
- browser test results
- before/after search behavior
- remaining risks
- screenshots if possible

Do not start with big refactor.

Do not change database schema.

Do not skip browser testing.

This is the right style: **small helper function** → **Docker check** → **browser test** → **next helper function**. No big “AI magic” rewrite.

Yes. Give Antigravity this. It has **50 small tasks** and tells you exactly when to browser test.

We will implement the buyer AI search upgrade in 50 very small tasks.

Main rule:

After every 5 tasks, stop and tell me: “READY FOR BROWSER TEST”.

Do not continue until I test or approve.

Global rules:

1. No Prisma schema change.
2. No major refactor.
3. No UI redesign.
4. Use existing Docker setup.
5. Use existing RunPod/Ollama setup.
6. Do not switch to local Ollama.
7. Keep changes small and reversible.
8. Commit only after browser test passes.

Goal:

Make buyer search reliable for:

“I want a black Mercedes S400”

Expected result:

Black Mercedes S400 should rank above white or silver Mercedes S400.

TASKS

1. Locate buyer search entry route.
2. Locate intent extraction function.
3. Locate SQL query builder.
4. Locate Qdrant search call.



5. Locate final ranking/merge logic.

STOP. READY FOR BROWSER TEST.

Browser test:

Search “black Mercedes S400” and record current result behavior.

6. Add traceSearchQuery(query) helper.
7. Log original user query.
8. Log extracted AI intent.
9. Log SQL filters.
10. Log Qdrant filters/results count.

STOP. READY FOR BROWSER TEST.

Browser test:

Search “black Mercedes S400” and confirm logs appear.

11. Add normalizeColor(value) helper.
12. Support black, white, silver, gray/grey.
13. Support red, blue, green, gold.
14. Support brown, beige, yellow.
15. Add unit tests or simple internal assertions for normalizeColor.

STOP. READY FOR BROWSER TEST.

Browser test:

Search black Mercedes, white Mercedes, gray car.

16. Add extractColorFromQuery(query).
17. Add extractBrandFromQuery(query).
18. Add extractModelFromQuery(query).
19. Add extractStorageFromQuery(query) for phones.
20. Add extractFurnishedFromQuery(query) for property.

STOP. READY FOR BROWSER TEST.

Browser test:

Search:

- black Mercedes S400
 - iPhone 15 Pro 256GB
 - furnished apartment Salmiya
21. Add extractAttributeTokens(query).
 22. Return color, brand, model, storage, furnished.
 23. Add price phrase detection like under/below/less than.
 24. Add simple location token detection.
 25. Log extracted rule-based attributes.

STOP. READY FOR BROWSER TEST.

Browser test:

Confirm extracted attributes appear in logs.



26. Add mergeAllIntentWithRuleIntent(aiIntent, ruleIntent).
27. Rule intent should fill missing AI fields.
28. AI intent should not be overwritten if already confident.
29. Log merged final intent.
30. Use merged intent in search path.

STOP. READY FOR BROWSER TEST.

Browser test:

Search “black Mercedes S400” and confirm final intent has color black, brand Mercedes, model S400.

31. Add buildSearchableAttributes(product).
32. Read from title.
33. Read from description.
34. Read from aiDescription.
35. Read from product.attributes JSON.

STOP. READY FOR BROWSER TEST.

Browser test:

Open existing listing/search and confirm attributes are built safely.

36. Extend buildSearchableAttributes to read vehicleDetails.
37. Extend buildSearchableAttributes to read propertyDetails.
38. Add safe fallback for missing attributes.
39. Ensure indexing never fails because attributes are missing.
40. Add logs for searchable attributes during indexing.

STOP. READY FOR BROWSER TEST.

Browser test:

Create a simple listing with only title and confirm it publishes.

41. Update Qdrant payload builder to include normalizedColor.
42. Add brand to Qdrant payload.
43. Add model to Qdrant payload.
44. Add condition/location/price to Qdrant payload.
45. Add searchableAttributes to Qdrant payload.

STOP. READY FOR BROWSER TEST.

Browser test:

Create Black Mercedes S400 listing and confirm Qdrant payload/log contains color black.

46. Add scoreAttributeMatch(intent, product).
47. Give color match +30.
48. Give brand match +25.
49. Give model match +25.
50. Apply attribute score to final ranking.

STOP. READY FOR BROWSER TEST.

Browser test:

Create:



- Black Mercedes S400
- White Mercedes S400
- Silver Mercedes S400

Search:

“black Mercedes S400”

Expected:

Black Mercedes S400 must rank first.

Final checks:

- docker compose up -d
- docker compose ps
- docker compose logs app --tail=100
- npm run typecheck
- npx prisma validate

Final report:

- Files changed
- Functions added
- Browser test results
- Before/after search result order
- Remaining risks



prompts



Use these one by one in Antigravity.

Prompt 1

Locate the buyer search entry route. Do not edit code. Report the file path and function flow.

Prompt 2

Locate the intent extraction function used by buyer search. Do not edit code. Report file path and inputs/outputs.

Prompt 3

Locate the SQL query builder for buyer listing search. Do not edit code. Report where filters are created.

Prompt 4

Locate where Qdrant/vector search is called. Do not edit code. Report the file path and function name.

Prompt 5

Locate final search ranking and result merge logic. Do not edit code. Then stop for browser test.

Browser Test 1

Run Docker, open browser, login, search "black Mercedes S400", and record current behavior.

Prompt 6

Add a small traceSearchQuery helper. It should accept query and log search debug info. Keep it minimal.

Prompt 7

Use traceSearchQuery to log the original user query only. Run typecheck.

Prompt 8

Extend traceSearchQuery to log extracted AI intent. Run typecheck.

Prompt 9

Extend traceSearchQuery to log SQL filters. Run typecheck.

Prompt 10



Extend `traceSearchQuery` to log Qdrant filters and result count. Run typecheck. Then stop for browser test.

Browser Test 2

Run Docker, search "black Mercedes S400", and confirm search trace logs appear.

Prompt 11

Add `normalizeColor(value)` helper. Support black and white only first. Run typecheck.

Prompt 12

Extend `normalizeColor` to support silver, gray, and grey. Run typecheck.

Prompt 13

Extend `normalizeColor` to support red, blue, green, and gold. Run typecheck.

Prompt 14

Extend `normalizeColor` to support brown, beige, and yellow. Run typecheck.

Prompt 15

Add simple internal tests/assertions for `normalizeColor`. Run tests/typecheck. Then stop for browser test.

Browser Test 3

Run Docker and search black Mercedes, white Mercedes, and gray car. Confirm color extraction/logs.

Prompt 16

Add `extractColorFromQuery(query)` using `normalizeColor`. Run typecheck.

Prompt 17

Add `extractBrandFromQuery(query)` for common brands: Mercedes, Toyota, BMW, Lexus, Nissan, Honda. Run typecheck.

Prompt 18

Add `extractModelFromQuery(query)` for model-like tokens such as S400, Camry, Land Cruiser, Patrol. Run typecheck.

Prompt 19



Add `extractStorageFromQuery(query)` for phone storage like 128GB, 256GB, 512GB, 1TB. Run typecheck.

Prompt 20

Add `extractFurnishedFromQuery(query)` for furnished and unfurnished property queries. Run typecheck. Then stop for browser test.

Browser Test 4

Search black Mercedes S400, iPhone 15 Pro 256GB, and furnished apartment Salmiya. Confirm extracted tokens in logs.

Prompt 21

Add `extractAttributeTokens(query)` that combines color, brand, model, storage, and furnished. Run typecheck.

Prompt 22

Make `extractAttributeTokens` return a clean object with missing fields omitted. Run typecheck.

Prompt 23

Add price phrase detection to `extractAttributeTokens`: under, below, less than. Run typecheck.

Prompt 24

Add simple location token detection from query. Keep it safe and minimal. Run typecheck.

Prompt 25

Log rule-based extracted attributes in search trace. Run typecheck. Then stop for browser test.

Browser Test 5

Search black Mercedes S400, Toyota under 3000, and furnished apartment Salmiya. Confirm rule attributes appear in logs.

Prompt 26

Add `mergeAIIntentWithRuleIntent(aiIntent, ruleIntent)`. Rule fields fill missing AI fields only. Run typecheck.

Prompt 27

Use `mergeAIIntentWithRuleIntent` in buyer search flow. Keep behavior unchanged except richer intent. Run typecheck.



Prompt 28

Ensure AI intent fields are not overwritten by rule intent when AI already has values. Run typecheck.

Prompt 29

Log the merged final intent in traceSearchQuery. Run typecheck.

Prompt 30

Use merged final intent for downstream SQL/Qdrant search. Run typecheck. Then stop for browser test.

Browser Test 6

Search "black Mercedes S400" and confirm final intent contains color black, brand Mercedes, model S400.

Prompt 31

Add buildSearchableAttributes(product) helper. Initially read only product title. Run typecheck.

Prompt 32

Extend buildSearchableAttributes to read product description. Run typecheck.

Prompt 33

Extend buildSearchableAttributes to read aiDescription. Run typecheck.

Prompt 34

Extend buildSearchableAttributes to read product.attributes JSON safely. Run typecheck.

Prompt 35

Add logs for buildSearchableAttributes output during search/indexing debug only. Run typecheck. Then stop for browser test.

Browser Test 7

Open/search existing listings and confirm searchable attributes are built safely without errors.

Prompt 36

Extend buildSearchableAttributes to read vehicleDetails safely if present. Run typecheck.

Prompt 37

Extend `buildSearchableAttributes` to read `propertyDetails` safely if present. Run typecheck.

Prompt 38

Add fallback so missing attributes never crash search or indexing. Run typecheck.

Prompt 39

Confirm indexing does not fail when title-only listing has missing attributes. Fix only if needed. Run typecheck.

Prompt 40

Add indexing debug log for searchable attributes. Run typecheck. Then stop for browser test.

Browser Test 8

Create a listing with only title. Confirm it publishes and indexing does not fail.

Prompt 41

Update Qdrant payload builder to include `normalizedColor` when available. Run typecheck.

Prompt 42

Update Qdrant payload builder to include `brand` when available. Run typecheck.

Prompt 43

Update Qdrant payload builder to include `model` when available. Run typecheck.

Prompt 44

Update Qdrant payload builder to include `condition`, `location`, and `price` when available. Run typecheck.

Prompt 45

Update Qdrant payload builder to include `searchableAttributes`. Run typecheck. Then stop for browser test.

Browser Test 9



Create Black Mercedes S400 listing and confirm logs/Qdrant payload include color black, brand Mercedes, model S400.

Prompt 46

Add scoreAttributeMatch(intent, product). Start with color scoring only. Run typecheck.

Prompt 47

Add +30 score for exact color match. Run typecheck.

Prompt 48

Add +25 score for exact brand match. Run typecheck.

Prompt 49

Add +25 score for exact model match. Run typecheck.

Prompt 50

Apply attribute score to final ranking. Run typecheck. Then stop for browser test.

Browser Test 10

Create Black, White, and Silver Mercedes S400 listings. Search "black Mercedes S400". Confirm black listing ranks first.



You've actually reached a very important milestone. The core AI marketplace search is no longer just keyword search — it's becoming an intent-aware marketplace search engine.

What is Completed

Infrastructure

- ✓ Production running on DigitalOcean
 - ✓ PostgreSQL
 - ✓ Redis
 - ✓ Qdrant
 - ✓ Dockerized deployment
 - ✓ RunPod GPU integration
 - ✓ Ollama integration
 - ✓ Qwen 14B
 - ✓ Nomic Embeddings
 - ✓ LLaVA Vision
-

Seller Flow

- ✓ User registration
- ✓ Login
- ✓ Create listing
- ✓ Edit listing
- ✓ Upload images
- ✓ AI-generated descriptions
- ✓ Category assignment
- ✓ Moderation pipeline



✓ Async indexing pipeline

✓ Qdrant indexing

AI Search Foundation

✓ Intent extraction

User can search:

Mercedes S400
Black Mercedes S400
BMW X5 blue
Villa in Salmiya

and the system extracts:

- Brand
- Model
- Color
- Year
- Category
- Condition

Search Improvements Completed

Fix 1

Before:

81 listings pending

After:

81 indexed

Database and Qdrant synchronized.

Fix 2

Qdrant payload enrichment.

Now Qdrant stores:

color

6/14/26 6:14:54 PM



```
make  
model  
year  
condition  
price  
price_band  
intent_tags  
smart_category
```

instead of basic data only.

Fix 3

SQL ranking.

Before:

```
Results sorted by created date
```

After:

```
Color match  
Brand match  
Model match  
Year match  
Condition match
```

affect ranking.

Fix 4

Search score.

Before:

```
searchScore = 0
```

After:

```
Black Mercedes S400 = 0.90
```

```
Mercedes S400 = 0.75
```

This is now visible to the API.

What Fahad Wants

Fahad keeps asking:

6/14/26 6:14:54 PM



Can I ask AI “I want a black Mercedes S400” and get the correct listing?

Status Today

Answer:

YES — partially working

Current flow:

black Mercedes S400

↓

Intent Extraction

↓

Brand = Mercedes

Model = S400

Color = Black

↓

SQL Search

↓

Attribute Ranking

↓

Mercedes S400 appears first

That part is working.

What Is NOT Finished Yet

1. Multiple Variant Testing

You only have one S400.

Need:

```text

Black S400

White S400

Silver S400



Then verify:

black Mercedes S400

returns Black first.

This is the next validation.

---

## 2. Qdrant Tier Usage

Current logic:

```
If SQL returns >= 3 results
Skip Qdrant
```

Meaning:

```
Tier 1 SQL
Tier 2 SQL
```

do most work.

Qdrant is often skipped.

We need to improve this.

Priority:



Medium

---

## 3. True Conversational Marketplace Search

Today:

black Mercedes S400

works.

Future:

```
I need a luxury black Mercedes under 20,000 KD
I want a family SUV with low mileage
Show me a villa with swimming pool in Salmiya
```

This is Fahad's real vision.

Priority:



High



---

## 4. Image-Based Search

Future:

User uploads image



LLaVA



Find similar products

Not built yet.

Priority:



Medium

---

## 5. Buyer AI Chat

Current:

Search API

Future:

User: I need a black Mercedes S400

AI:

I found 3 options:

1. Mercedes S400 2020
2. Mercedes S400 2019
3. Mercedes S400 2021

Which one would you like?

This is the real marketplace assistant.

Priority:



Very High

---

## 6. Admin AI Monitoring Dashboard



Still pending.

Need:

- Moderation history
- Failed jobs
- Embedding failures
- Agent logs
- Queue status
- Reindex button

Priority:



Medium

---

## My Assessment

If we look at the entire SellSell / PickPic roadmap:

### Backend Infrastructure

~90% complete

### Seller Flow

~85% complete

### AI Search Engine

~75% complete

### Buyer AI Assistant

~35% complete

### Admin AI Dashboard

~20% complete

### Mobile App

~15% complete

---

## Most Important Next Step



If I were prioritizing tomorrow's work, I would do:

## Phase A

Create test listings:

Black Mercedes S400  
White Mercedes S400  
Silver Mercedes S400

Verify ranking.

## Phase B

Remove excessive Qdrant skipping and improve hybrid ranking.

## Phase C

Build the conversational buyer assistant that Fahad keeps asking about:

I need a black Mercedes S400

↓

AI understands intent

↓

Searches listings

↓

Returns marketplace recommendations

↓

Asks follow-up questions

Once Phase C is done, Fahad will finally see the product behaving like the AI marketplace he's been describing since day one.